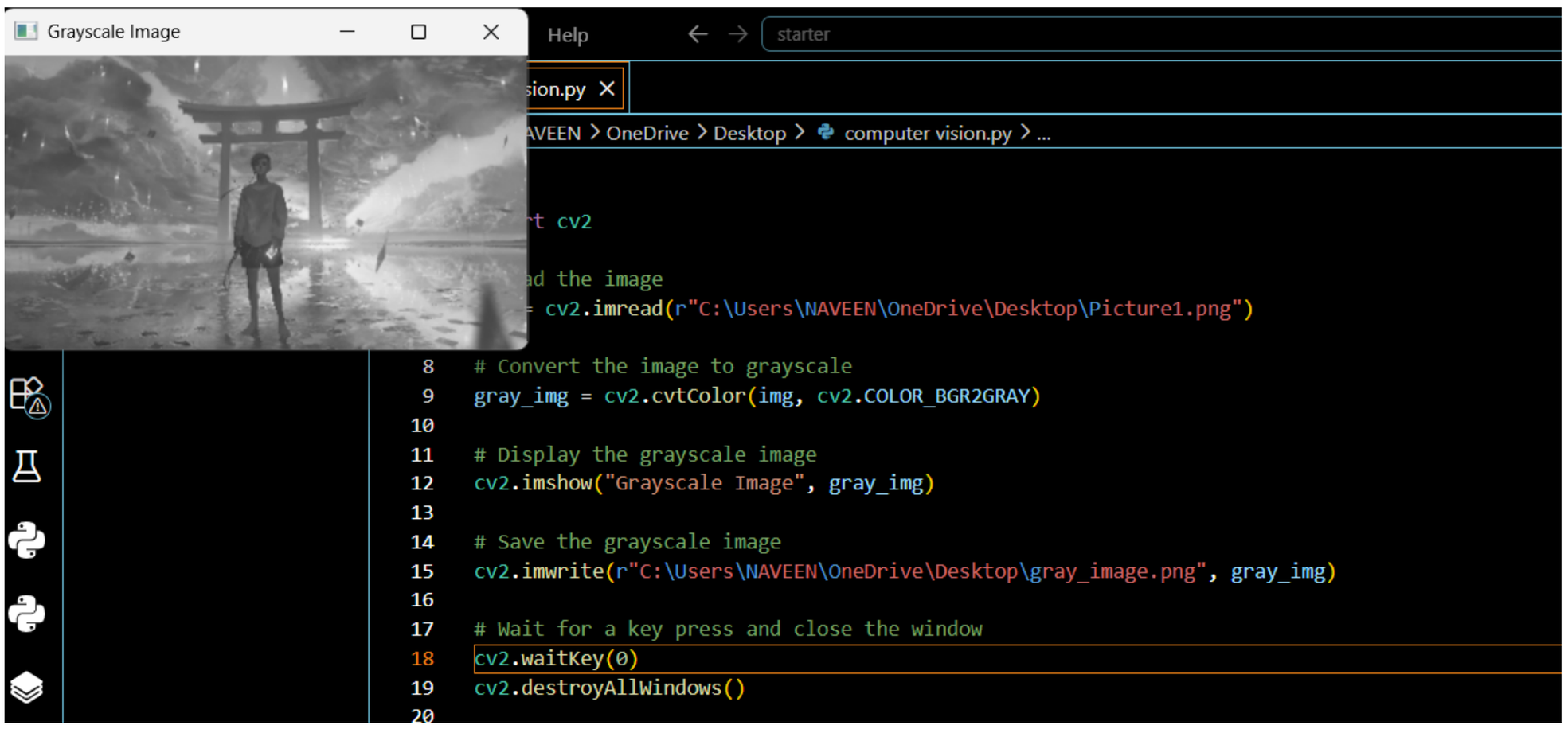
1.Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

```python
import cv2

# Read the image
img = cv2.imread(r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png")

# Convert the image to grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display the grayscale image
cv2.imshow("Grayscale Image", gray_img)

# Save the grayscale image
cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\gray_image.png", gray_img)

# Wait for a key press and close the window
cv2.waitKey(0)
cv2.destroyAllWindows()
```



2.Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

```python
import cv2
img = cv2.imread(r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Canny_Edges_3rd.jpg", edges)
cv2.imshow('Edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3.Perform Harris corner detection on images containing varying textures and patterns.

Explain the concept of corner detection and its importance in computer vision tasks.

Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

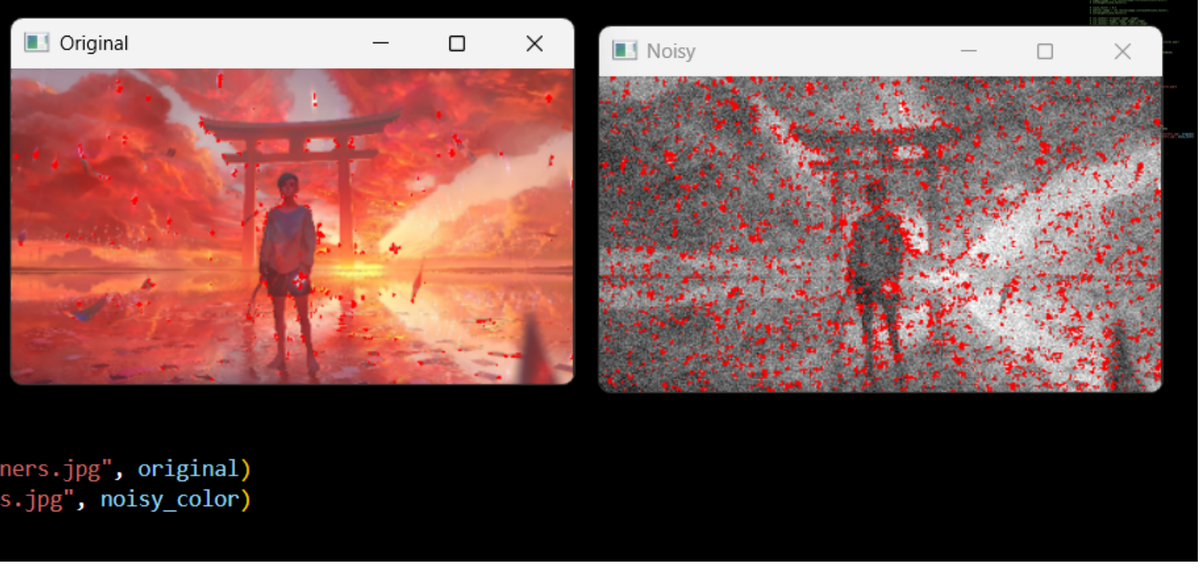
```python
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png", cv2.IMREAD_GRAYSCALE)
kernel = np.ones((5,5), np.uint8)
dilated_img = cv2.dilate(img, kernel, iterations=1)
cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Dilated_Image.jpg", dilated_img)

cv2.imshow('Dilated Image', dilated_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

4. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

```python
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

noise = np.random.normal(0, 25, gray.shape)
noisy = np.clip(gray + noise, 0, 255).astype(np.uint8)

gray_float = np.float32(gray)
noisy_float = np.float32(noisy)

corners1 = cv2.cornerHarris(gray_float, 2, 3, 0.04)
corners2 = cv2.cornerHarris(noisy_float, 2, 3, 0.04)

original = img.copy()
noisy_color = cv2.cvtColor(noisy, cv2.COLOR_GRAY2BGR)

original[corners1 > 0.01 * corners1.max()] = [0, 0, 255]
noisy_color[corners2 > 0.01 * corners2.max()] = [0, 0, 255]

cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Original_Corners.jpg", original)
cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Noisy_Corners.jpg", noisy_color)

cv2.imshow("Original", original)
cv2.imshow("Noisy", noisy_color)

cv2.waitKey(0)
cv2.destroyAllWindows()
```


```python
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

noise = np.random.normal(0, 25, gray.shape)
noisy = np.clip(gray + noise, 0, 255).astype(np.uint8)

gray_float = np.float32(gray)
noisy_float = np.float32(noisy)

corners1 = cv2.cornerHarris(gray_float, 2, 3, 0.04)
corners2 = cv2.cornerHarris(noisy_float, 2, 3, 0.04)

original = img.copy()
noisy_color = cv2.cvtColor(noisy, cv2.COLOR_GRAY2BGR)

original[corners1 > 0.01 * corners1.max()] = [0, 0, 255]
noisy_color[corners2 > 0.01 * corners2.max()] = [0, 0, 255]

cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Original_Corners.jpg", original)
cv2.imwrite(r"C:\Users\NAVEEN\OneDrive\Desktop\Noisy_Corners.jpg", noisy_color)
```

5. Hough Transform for Shape Detection

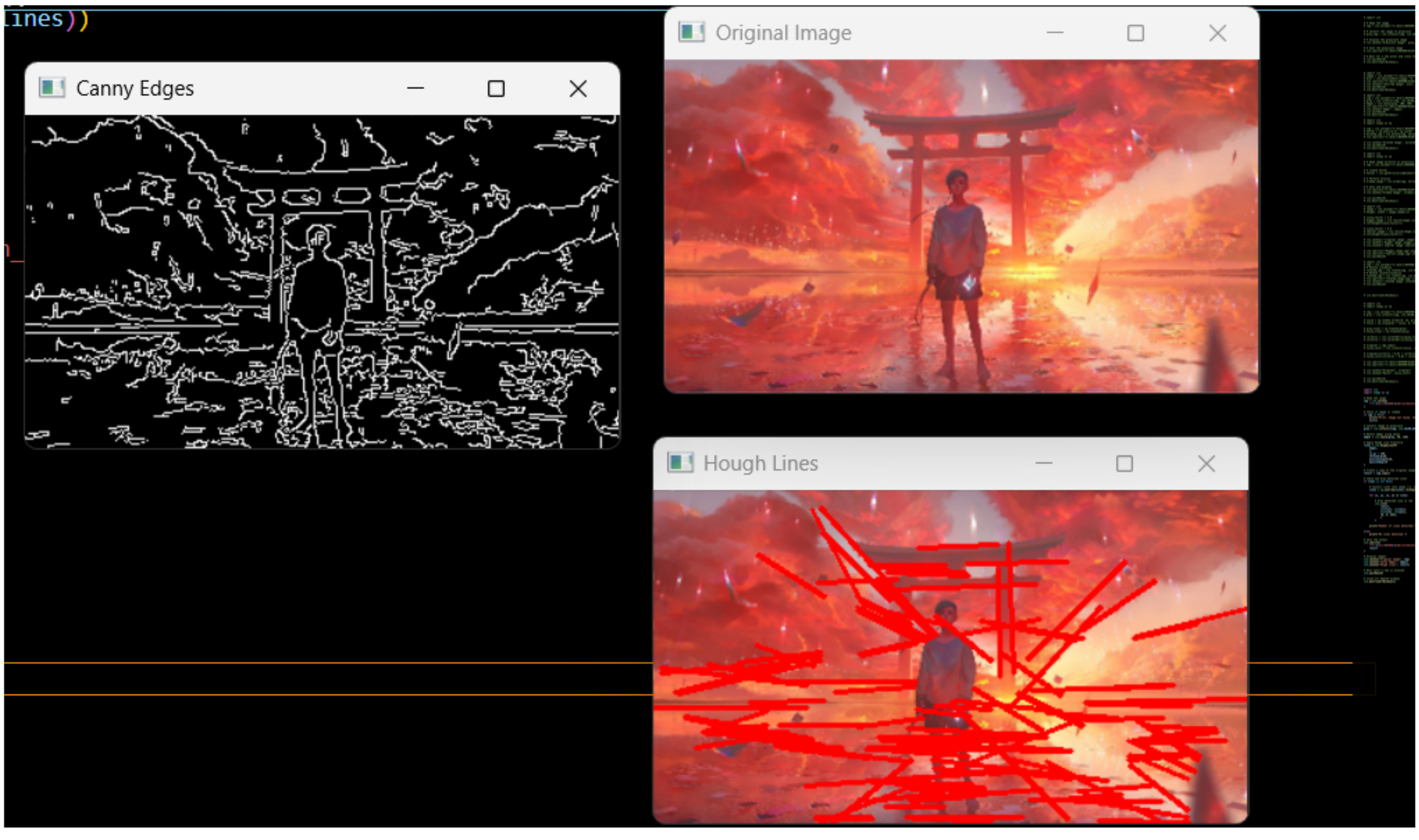
Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

```
import cv2
import numpy as np


# Read the image
img = cv2.imread(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png"
)


# Check if image is loaded
if img is None:
    print("Error: Image not found. Check the file path.")
    exit()


# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


# Detect edges using Canny
```

```
edges = cv2.Canny(gray, 50, 150)


# Apply Hough Line Transform
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=50,
    minLineLength=30,
    maxLineGap=10
)


# Create a copy of the original image
result = img.copy()


# Check and draw detected lines
if lines is not None:


    # Convert lines into shape (-1, 4)
    lines = np.asarray(lines).reshape(-1, 4)


    for x1, y1, x2, y2 in lines:


        # Draw detected line in red
        cv2.line(
            result,
```

```
        (int(x1), int(y1)),
        (int(x2), int(y2)),
        (0, 0, 255),
        2
    )

    print("Number of lines detected:", len(lines))

else:
    print("No lines detected.")

# Save the output
cv2.imwrite(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Hough_Lines.jpg",
    result
)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Canny Edges", edges)
cv2.imshow("Hough Lines", result)

# Wait until a key is pressed
cv2.waitKey(0)

# Close all OpenCV windows
```

cv2.destroyAllWindows()



6. Effect of Smoothing on Edge Detection

Design and perform an experiment to study the impact of smoothing techniques on edge detection performance. Explain the theoretical purpose of smoothing in reducing noise and improving edge continuity. Apply smoothing and edge detection methods using appropriate tools, document outputs before and after smoothing systematically, and analyze how smoothing affects edge clarity, continuity, and noise suppression.

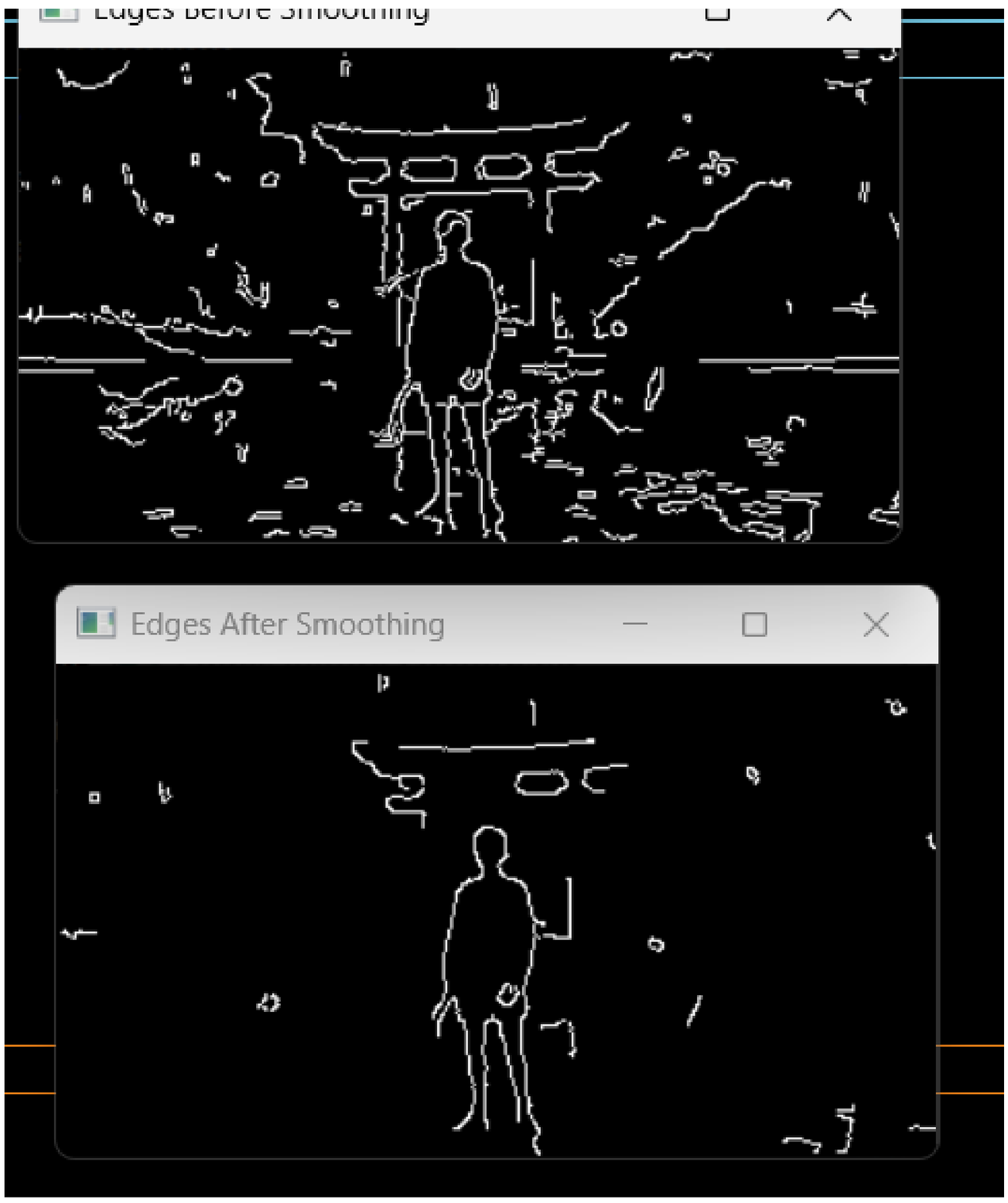
import cv2

Read image

img = cv2.imread(

```
    r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png",
    cv2.IMREAD_GRAYSCALE
)

if img is None:
    print("Error: Image not found.")
    exit()

# Edge detection without smoothing
edges_before = cv2.Canny(img, 100, 200)

# Apply Gaussian smoothing
blur = cv2.GaussianBlur(img, (5, 5), 0)

# Edge detection after smoothing
edges_after = cv2.Canny(blur, 100, 200)

# Save results
cv2.imwrite(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Edges_Before_Smoothing.jpg",
    edges_before
)

cv2.imwrite(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Edges_After_Smoothing.jpg",
    edges_after
```

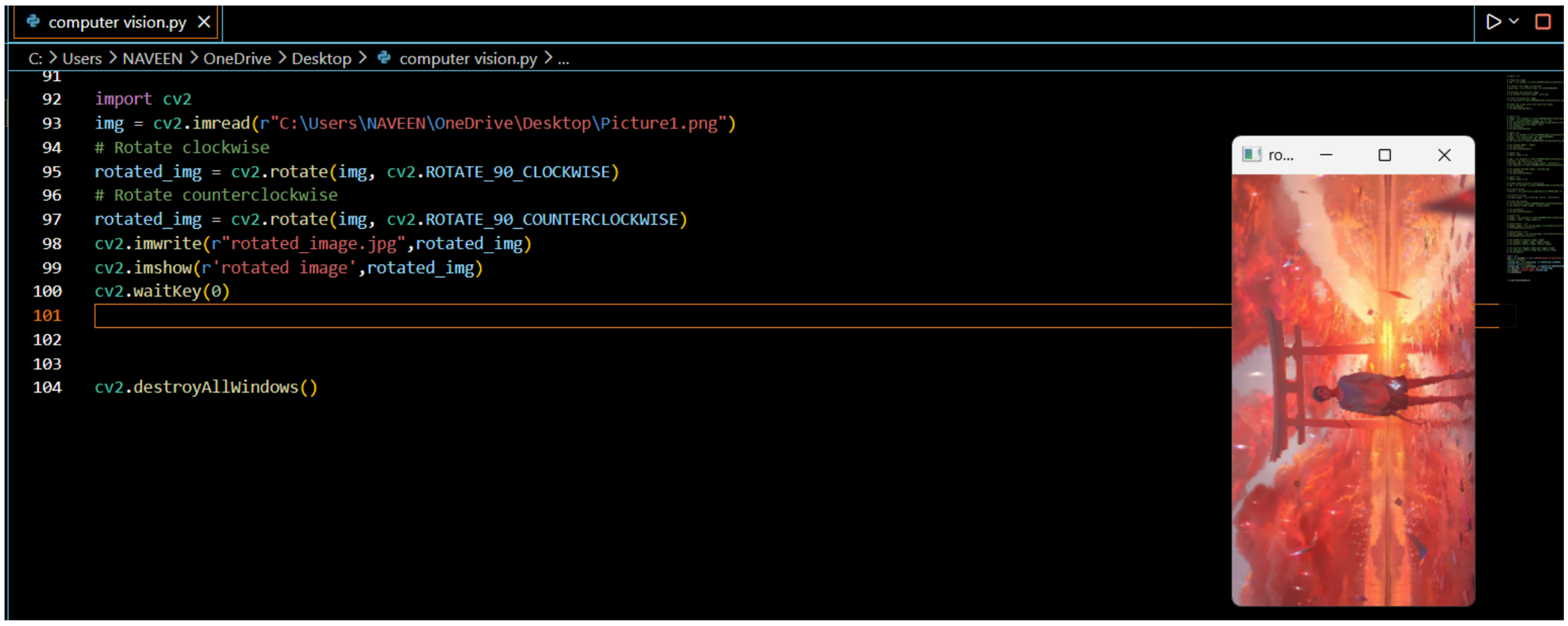
```
)

# Display results
cv2.imshow("Edges Before Smoothing", edges_before)
cv2.imshow("Edges After Smoothing", edges_after)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



7. Rotation Impact on Feature Matching

Conduct an experiment to evaluate the effect of image rotation on feature matching accuracy. Explain the significance of rotation invariance in feature matching

algorithms. Rotate images at different angles, perform feature matching using suitable tools, document matching results clearly, and analyze the robustness of the method against rotational transformations.



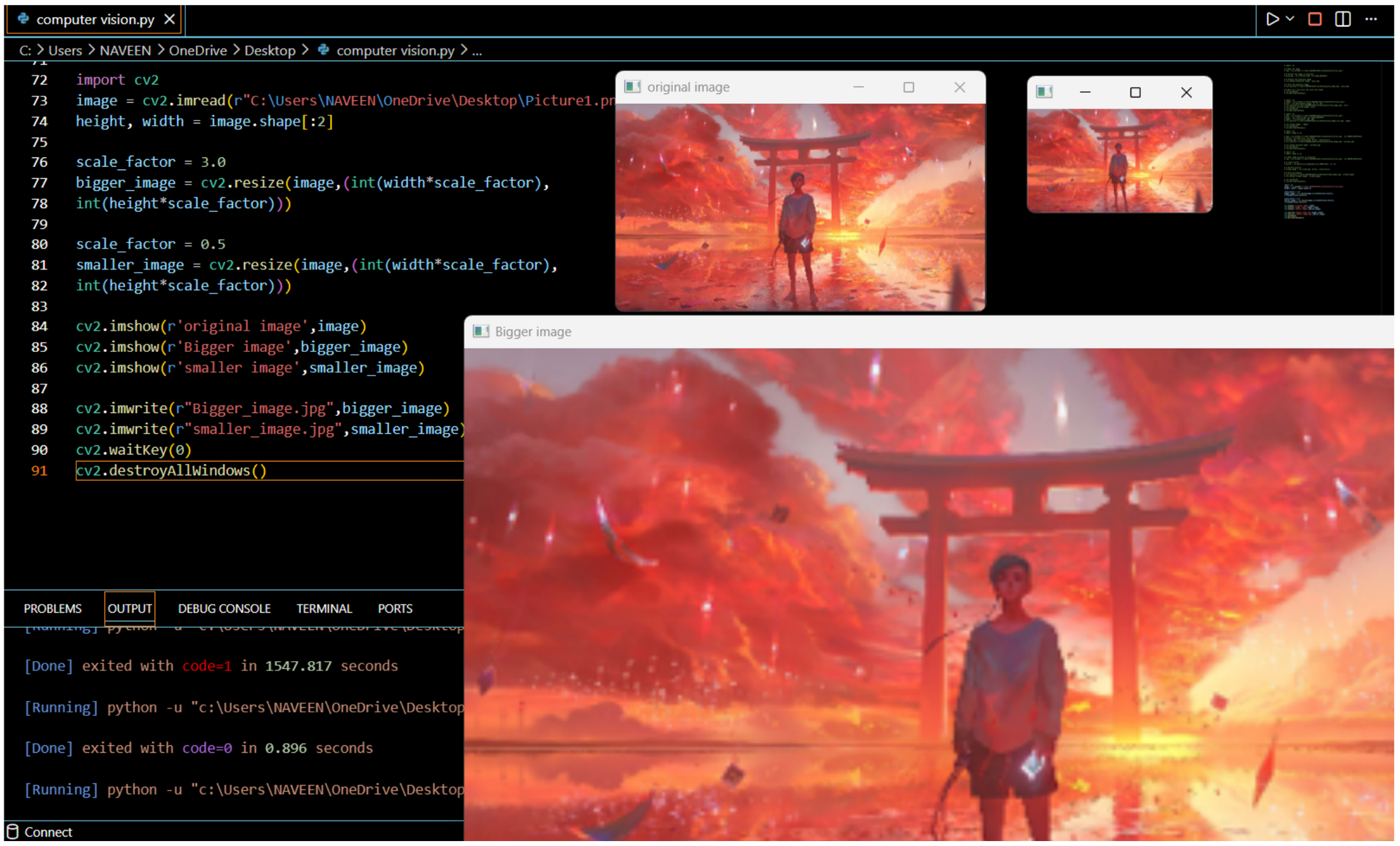
8. Effect of Blurring on Feature Detection

Conduct an experiment to study the influence of image blurring on feature detection. Explain the purpose of blurring and its effect on image details. Apply blur filters followed by feature detection techniques, document the outputs clearly, and analyze how blurring affects feature visibility, accuracy, and robustness.

9. Effect of Image Scaling on Feature Detection

Perform an experiment to analyze how image scaling affects feature detection accuracy. Explain the importance of scale in computer vision and feature extraction

tasks. Resize images to different scales, apply feature detection techniques, document detected features systematically, and analyze the impact of scaling on feature consistency and detection performance.



10. Threshold Effect in Edge Detection

Design an experiment to study the influence of threshold values on edge detection quality. Explain the importance of thresholding in distinguishing meaningful edges from noise. Apply edge detection using different threshold values, record the outputs systematically, and analyze how threshold selection affects edge continuity, accuracy, and noise sensitivity.

import cv2

Read image

```
img = cv2.imread(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Picture1.png",
    cv2.IMREAD_GRAYSCALE
)

if img is None:
    print("Error: Image not found.")
    exit()

# Edge detection with different threshold values
edges1 = cv2.Canny(img, 50, 100)
edges2 = cv2.Canny(img, 100, 200)
edges3 = cv2.Canny(img, 150, 250)

# Save outputs
cv2.imwrite(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Edges_50_100.jpg",
    edges1
)

cv2.imwrite(
    r"C:\Users\NAVEEN\OneDrive\Desktop\Edges_100_200.jpg",
    edges2
)

cv2.imwrite(
```

```
    r"C:\Users\NAVEEN\OneDrive\Desktop\Edges_150_250.jpg",
    edges3
)

# Display outputs
cv2.imshow("Threshold 50-100", edges1)
cv2.imshow("Threshold 100-200", edges2)
cv2.imshow("Threshold 150-250", edges3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```